Important points

1) If you are on windows os then to execute git commands "git bash" is required.

2) We need Linux like enviroment to execute git command, bydefault Linux enviroment is not available on windows so get Linux like enviroment on windows we install git bash.
i.e git bash is providing linux enviroment for us.

3) When you install "git bash" software then internally three software get installed on your system and these softwares are - Git software,Git CLI and Git GUI
i.e Git Bash comes with Git Software + Git CLI + Git GUI

4) CLI and GUI are the interfaces to interact with "git software"

5) some git commands are simillar to Linux , only difference is
git command start with "git" and linux command not start with git
-> IF any command start with git then it is considered as git command.
-> If any command not starting with git then it is considered as Linux command

Some examples
1) cd
2) ls
3) pwd
4) git config --list
5) git init
6) rm File_name
7) git rm --cached File_name

6) Linux don't have drive concept,in linux everything is treated as File.
7) In Linux File extentions like .txt .pdf is not import, Linux decides file type based on its content.
8) File name is case sensitive
F1.txt
f1.txt
above files are not same,

9) Bydefault Empty Folder is not traced by git
-> If we create an empty folder and check the status of repository then that empty folder is not showed in the status

git rm command

-> Here rm stands for remove.
-> git has rm command and linux also have rm command.

-> Usage of rm command
1) If we use "rm" command as Linux command then it will delete a file from our system.
i.e file will be deleted from our project.

2) If we use "rm" command as git command then
1) it will remove a file from staging area and put back into working area.
i.e only unstage a file not deletion
2) First Remove from staging area and delete from working area
i.e staging + deletion at a time

Case 1: If file is present in working area and you want to delete from system then use linux "rm" command

Syntax 1: rm Fil1_name Fil2_name File3_name ....

Syntax 2: rm -f Fil1_name Fil2_name File3_name ....

Here -f stands for forcefully removal.
Here rm command is considered as Linux command not git command.

Case 2: If file is present in staging area and you want to remove a file from staging area and put back into working area then use "git rm" command with --cached option.

Syntax: git rm --cached Fil1_name Fil2_name File3_name ...

Case 3: If file is present in staging area and you want to remove from staging area and delete from system then use "git rm" command with -f option.

Syntax: git rm -f Fil1_name Fil2_name File3_name ....

Case 4: If there is a directory present in working area and you want to delete from system then use linux "rm" command with -r option.

Syntax : rm -r directory_name
Here -r stands for Recursivly

Case 5: If there is a directory present in staging area and you want to remove this directory from staging area and place back into working area then use "git rm" command with --cached and -r option

Syntax: git rm --cached -r directory_name
Here oder of option is not important.

git rm --cached -r directory_name
or
git rm -r --cached directory_name
These two Syntax are same.

Case 6: If there is a directory present in staging area and you want to remove from staging area and delete from Syntax then use "git rm" command with -rf option

Syntax: git rm -rf directory_name

or

Syntax: git rm -r -f directory_name

Order of option is not important, we can use -r first or -f

Conclusions:

1) If file is present in working area and you want to delete the use linux rm command

2) If file is present in staging area and you want delete from staging area and working area then use "git rm" command with -f option

3) If file is present in staging area and you want to remove from staging area but keep in working area then use "git rm" command with --cached option.

4) If there is directory present in working area and you want to delete from system then use linux "rm" command with -r option

5) If there is directory, present in staging area and you want to delete from staging area and also delete from working area then use "git rm" command with "-rf" option

6) If there is directory present in staging area and you want to remove from staging area but keep in working area then use "git rm" command with --cached and -r option

7) If rm command start with "git" then it is treated as "git" command
8) If rm command not starting with "git" then it is treated as linux command

Q. What is mean by Untracked File?
-> Newly added file or the file whose record is not present in git is considered as Untracked file.Git bash display untracked file name in red color.

Q. What is mean by Tracked file or staged file?
-> The file which is present in staging area is considered as staged file or Tracked file.Git bash display Tracked file name in green color.

Q. What is mean by Commited File?
-> The File which is present in local repository(commit area) is considered as Commited File.

4) What is mean by Modified File?
-> The file whose contents are modified after pushing it into staging area or commit area is considered are as modified file.
Git bash display modified file name in red color.

-> To commit(save) the changes we need to perform below operations on that file
1) First- Add it into staging area by using "git add" command
2) Second- Add it into commit area by "git commit" command.

git restore command

-> Usage of git restore command
1) The "git restore" command is used to restore file content to a previous state.
2) The "git restore" command is used to unstage a file.

Usage 1:Restore file content to a previous state.
If you have modified a file and want to discard those changes and restore the file to the last committed state then use "git restore" command

Case 1: Restore single file changes to previous state.
Syntax: git restore Fil1_name
Example: git restore F1.txt

Case 2: Restore multiple file changes to previous state
Syntax: git restore Fil1_name Fil2_name File3_name ...
Example: git restore F1.txt F2.txt F3.txt

Case 3:Restore All Files at a time
Syntax: git restore .
If we want to restore all files at a time use dot symbol after "restore"
Example: git restore .

Case 4: Restore to a Specific Commit:
Syntax: git restore --source=<commit-hash> FileName
Note: When we commit the changes then unique commit id is generated this considered as commit-hash.

Usage 2:unstage a file using "git restore"
If you have added a file in the staging area by using "git add" command but you want to unstage that file then use "git restore" command with --staged option.

Case 1: Unstage single file at a time
Syntax: git restore --staged Fil1_name

Usage 2:unstage a file using "git restore"

If you have added a file in the staging area by using "git add" command but you want to unstage that file then use "git restore" command with --staged option.

Case 1: Unstage single file at a time

Syntax: git restore --staged Fil1_name

Example: git restore --staged F1.txt

Case 2: Unstage multiple files at a time

Syntax: git restore --staged Fil1_name Fil2_name File3_name ...

Example: git restore --staged F1.txt F2.txt F3.txt

Case 3: Unstage all files at a time

Syntax: git restore --staged .

Example: git restore --staged .

Here we have do symbol

Conclusions

1) "git restore" command with --staged" option unstage the file without discarding
2) "git restor" command without --staged option discard the changes of working area files.
3) If you want to discard the staging area file changes then first unstage the file and then discard the changes.

Important points:

We can unstage a file using "git rm --cached" command and "git restore --staged" command.

When we should go with "git rm --cached" and when we should go with "git restore --staged" command?

-> If untracked file is added into staging area and you want to unstage then we can use either "git rm" or git restore command but commited file is modified and added its changes to staging area and you want unstage then use "git restore --staged"